Assembly Language

A Beginner's Guide to x86-64 Programming

Page	Topic
2	What is Assembly Language?
3	Number Systems and Data
4	CPU Registers
5	Basic Instructions
6	Addressing Modes
7	Flags and Control Flow
8	The Stack and Procedures
9	Example Programs
10	System Calls, Tools and Summary

Syntax used: NASM (Intel style) on 64-bit Linux.

1. What is Assembly Language?

Assembly language is a low-level programming language in which each statement corresponds closely to a single machine instruction executed by the CPU. Instead of writing binary opcodes such as `10110000 01100001`, a programmer writes readable mnemonics such as `mov al, 97`. A program called an **assembler** translates the mnemonics into machine code.

Language Levels

Level	Example	Notes
High-level	Python, Java, C#	Portable, abstract, easy to write
Mid-level	C, C++, Rust	Close to hardware yet portable
Assembly	NASM, MASM, GAS	One instruction per line, CPU specific
Machine code	0xB0 0x61	Raw binary the CPU executes

Why Learn Assembly?

- **Understand the machine:** see how memory, registers and the stack really work.
- **Performance:** hand-tuned routines in games, codecs and cryptography.
- **Embedded systems:** bootloaders, interrupt handlers and tiny microcontrollers.
- **Security:** reverse engineering, malware analysis and debugging crashes.
- **Compilers and OS:** required knowledge for writing kernels and compilers.

Assembler Families

Assembly is specific to an **instruction set architecture (ISA)**. The same idea of adding two numbers is written differently on x86-64, ARM or RISC-V. Even within x86 there are two syntax styles: **Intel** syntax (`mov rax, 5`, destination first) and **AT&T** syntax (`movq $5, %rax`, source first). Popular assemblers include NASM, MASM, FASM and GNU as (GAS).

The Build Process

Source (`.asm`) is assembled into an object file (`.o`) and then linked by a linker into an executable. The CPU runs this in a continuous fetch, decode, execute cycle: fetch the instruction at the address in the instruction pointer, decode it, execute it, and advance.

2. Number Systems and Data

Computers store everything as bits. Assembly programmers constantly switch between **binary**, **decimal** and **hexadecimal**. Hex is popular because one hex digit equals exactly four bits.

Decimal	Binary	Hex
0	0000	0
5	0101	5
10	1010	A
15	1111	F
255	1111 1111	FF

Data Sizes

Name	Bits	Bytes	NASM directive
Byte	8	1	db
Word	16	2	dw
Double word	32	4	dd
Quad word	64	8	dq

Signed Numbers: Two's Complement

Negative integers use **two's complement**: invert all bits and add 1. For an 8-bit value, +5 is `00000101`, so -5 is `11111011`. An 8-bit signed number ranges from -128 to +127, while unsigned covers 0 to 255. The CPU uses the same add instruction for both; the interpretation is up to the programmer.

Endianness

x86 is **little-endian**: the least significant byte is stored at the lowest address. The 32-bit value `0x12345678` is stored in memory as bytes `78 56 34 12`.

Characters and Strings

Text is stored as numeric codes, typically ASCII. For example 'A' is 65 (0x41) and 'a' is 97 (0x61). A C-style string ends with a zero byte.

```
section .data
    num   dd 100          ; 32-bit integer
    msg   db "Hello", 0   ; zero-terminated string
    arr   dw 1, 2, 3, 4   ; array of 16-bit words
section .bss
    buf   resb 64         ; reserve 64 uninitialised bytes
```

3. CPU Registers

Registers are tiny, extremely fast storage locations inside the CPU. x86-64 provides sixteen 64-bit general purpose registers. The lower portions can be accessed under smaller names.

64-bit	32-bit	16-bit	8-bit	Common role
RAX	EAX	AX	AL	Accumulator, return value
RBX	EBX	BX	BL	Base, callee-saved
RCX	ECX	CX	CL	Counter, 4th argument
RDX	EDX	DX	DL	Data, 3rd argument
RSI	ESI	SI	SIL	Source index, 2nd argument
RDI	EDI	DI	DIL	Destination index, 1st argument
RSP	ESP	SP	SPL	Stack pointer
RBP	EBP	BP	BPL	Base (frame) pointer
R8-R15	R8D-R15D	R8W-R15W	R8B-R15B	Extra registers, R8 and R9 are args 5 and 6

Special Registers

- **RIP** (instruction pointer): address of the next instruction to execute.
- **RFLAGS**: status bits such as zero, carry, sign and overflow, set by arithmetic and compare operations.
- **XMM0-XMM15**: 128-bit registers for floating point and SIMD operations.
- **Segment registers** (CS, DS, SS, ...): largely unused in 64-bit flat mode.

Partial Register Behaviour

Writing to a 32-bit register (such as `eax`) automatically zeroes the upper 32 bits of the 64-bit register. Writing to 8-bit or 16-bit parts leaves the other bits unchanged. This is why `xor eax, eax` is the standard, compact way to set RAX to zero.

```
mov rax, 0x1122334455667788
mov eax, 5          ; rax = 0x0000000000000005 (upper half cleared)
mov al,  0xFF       ; only the lowest byte changes
```

4. Basic Instructions

An instruction has a **mnemonic** and zero to three **operands**. In Intel syntax the destination comes first: `mnemonic destination, source`. Operands may be registers, memory locations or immediate (constant) values. Two memory operands in one instruction are not allowed.

Data Movement

Instruction	Meaning	Example
mov	Copy source to destination	mov rax, rbx
lea	Load effective address (no memory access)	lea rax, [rbx+rcx*4]
push / pop	Push to / pop from the stack	push rax
xchg	Swap two operands	xchg rax, rbx
movzx / movsx	Move with zero / sign extension	movzx eax, byte [rsi]

Arithmetic and Logic

Instruction	Meaning	Example
add / sub	Addition / subtraction	add rax, 10
inc / dec	Add or subtract 1	inc rcx
mul / imul	Unsigned / signed multiply	imul rax, rbx
div / idiv	Unsigned / signed divide (RDX:RAX / operand)	div rcx
neg	Two's complement negation	neg rax
and / or / xor / not	Bitwise operations	xor eax, eax
shl / shr / sar	Shift left / right (logical, arithmetic)	shl rax, 3
cmp / test	Compare / AND without storing result	cmp rax, 5

Quick Example

```
; compute (a + b) * 2 - 1
mov rax, 7
add rax, 3        ; rax = 10
shl rax, 1        ; rax = 20  (shift left 1 = multiply by 2)
dec rax           ; rax = 19
```

Tip: shifting left by n multiplies by 2^n, and is usually faster than `mul`.

5. Addressing Modes

An addressing mode describes how an instruction finds its operand. Square brackets mean *the contents of the memory at that address*. The general x86 form is:

```
[ base + index * scale + displacement ]
   base, index = registers     scale = 1, 2, 4 or 8     displacement = constant
```

Mode	Example	Description
Immediate	mov rax, 42	Constant value is part of the instruction
Register	mov rax, rbx	Operand is a register
Direct	mov rax, [value]	Memory at a fixed label/address
Register indirect	mov rax, [rbx]	Address held in a register (like a pointer)
Base + displacement	mov rax, [rbx+8]	Struct field access
Indexed + scale	mov rax, [rbx+rcx*8]	Array element access
RIP-relative	mov rax, [rel value]	Position-independent code (default in 64-bit)

Size Specifiers

When the size cannot be inferred from a register, it must be stated with `byte`, `word`, `dword` or `qword`:

```
mov byte  [buf], 65      ; store 1 byte
mov dword [num], 100     ; store 4 bytes
mov qword [rbx+8], 0     ; store 8 bytes
```

Array Traversal Example

Summing a quad-word array using an indexed addressing mode:

```
section .data
    arr dq 10, 20, 30, 40
section .text
    xor rax, rax            ; sum = 0
    xor rcx, rcx            ; i = 0
.loop:
    add rax, [arr + rcx*8]  ; sum += arr[i]
    inc rcx
    cmp rcx, 4
    jl  .loop               ; repeat while i < 4   (rax = 100)
```

6. Flags and Control Flow

By default the CPU executes instructions in sequence. **Jump** instructions change RIP to implement decisions and loops. Conditional jumps examine **flags** set by the previous arithmetic or `cmp` instruction.

Flag	Name	Set when
ZF	Zero	Result is zero (or operands equal after cmp)
SF	Sign	Result is negative (top bit is 1)
CF	Carry	Unsigned overflow / borrow
OF	Overflow	Signed overflow

Jump Instructions

Instruction	Condition	Type
jmp	Always	Unconditional
je / jne	Equal / not equal (ZF)	Both
jg / jge / jl / jle	Greater / less (signed)	Signed
ja / jae / jb / jbe	Above / below (unsigned)	Unsigned
jz / jnz	Zero / not zero	Flag-based
loop	Decrement RCX, jump if not zero	Loop

If / Else

```
; if (rax > 10) rbx = 1; else rbx = 0;
    cmp rax, 10
    jle .else
    mov rbx, 1
    jmp .done
.else:
    mov rbx, 0
.done:
```

Counting Loop

```
; for (i = 5; i > 0; i--) total += i;
    mov rcx, 5
    xor rax, rax
.again:
    add rax, rcx
    dec rcx
    jnz .again            ; rax = 15 when finished
```

7. The Stack and Procedures

The **stack** is a region of memory that grows toward lower addresses and is managed through RSP. `push` decrements RSP by 8 and stores a value; `pop` loads a value and increments RSP by 8. The stack holds return addresses, saved registers and local variables.

Calling and Returning

`call label` pushes the return address and jumps to the label. `ret` pops that address back into RIP.

```
    mov rdi, 6
    call square         ; result returned in rax
    ...
square:
    mov rax, rdi
    imul rax, rdi       ; rax = rdi * rdi
    ret
```

System V AMD64 Calling Convention (Linux, macOS)

Item	Rule
Integer arguments	RDI, RSI, RDX, RCX, R8, R9, then the stack
Return value	RAX (and RDX for 128-bit results)
Caller-saved	RAX, RCX, RDX, RSI, RDI, R8-R11
Callee-saved	RBX, RBP, R12-R15 (must be preserved)
Alignment	RSP must be 16-byte aligned at the point of a call

Stack Frame Template

```
my_func:
    push rbp              ; save caller's frame pointer
    mov  rbp, rsp         ; establish new frame
    sub  rsp, 16          ; reserve local variable space
    mov  qword [rbp-8], rdi   ; local variable
    ; ... function body ...
    mov  rsp, rbp         ; (or: leave)
    pop  rbp
    ret
```

8. Example Programs

Program 1: Hello, World (Linux x86-64)

```
section .data
    msg db "Hello, World!", 10      ; 10 = newline
    len equ $ - msg                 ; length computed by assembler

section .text
    global _start
_start:
    mov rax, 1          ; syscall number: write
    mov rdi, 1          ; file descriptor: stdout
    mov rsi, msg        ; buffer address
    mov rdx, len        ; number of bytes
    syscall

    mov rax, 60         ; syscall number: exit
    xor rdi, rdi        ; exit code 0
    syscall
```

Program 2: Factorial (iterative)

```
; input: rdi = n      output: rax = n!
factorial:
    mov rax, 1
.next:
    cmp rdi, 1
    jle .end
    imul rax, rdi       ; rax *= n
    dec rdi
    jmp .next
.end:
    ret                 ; factorial(5) returns 120
```

Program 3: String Length

```
; input: rdi = address of zero-terminated string
; output: rax = length
strlen:
    xor rax, rax
.scan:
    cmp byte [rdi+rax], 0
    je  .done
    inc rax
    jmp .scan
.done:
    ret
```

Each routine follows the calling convention from the previous chapter, so they can be called from C or from other assembly code.

9. System Calls, Tools and Summary

System Calls

To request services from the operating system, a program places the syscall number in RAX, arguments in RDI, RSI, RDX, R10, R8, R9, and executes `syscall`. The result is returned in RAX.

Number	Name	Purpose
0	read	Read from a file descriptor
1	write	Write to a file descriptor
2	open	Open a file
3	close	Close a file
60	exit	Terminate the process

Building and Debugging

```
nasm -f elf64 hello.asm -o hello.o     # assemble
ld hello.o -o hello                    # link
./hello                                # run

gdb ./hello                            # debug
(gdb) break _start
(gdb) run
(gdb) stepi                            # execute one instruction
(gdb) info registers                   # inspect registers
objdump -d -M intel hello              # disassemble
```

Summary of Key Points

- Assembly maps almost one-to-one onto machine instructions for a specific CPU.
- Registers hold working data; memory is accessed with flexible addressing modes.
- Flags set by `cmp` and arithmetic drive conditional jumps.
- The stack supports function calls, local variables and saved state.
- A calling convention lets separately written routines work together.
- System calls connect your program to the operating system.

Where to Go Next

Practice by writing small programs: reverse a string, convert a number to text, implement Fibonacci, or call C library functions such as `printf` from assembly. Then read compiler output (`gcc -S -masm=intel`) to see how high-level code is translated.